

DressMe

Phase 3 — Data collection & Exploratory Data Analysis

AI personal fashion assistant · ESPRIT, Advanced Data Science project (team of 6) · Report generated 2026-10-07

Executive summary

- Three public datasets were explored and mapped to one **unified label schema**: Kaggle Fashion Product (35,432 rows kept), Fashionpedia (160,080 worn items) and Maryland PolyVore (125,910 product shots in 31k outfits): **321,422 labelled items** in total.
- The sources complement each other: Fashion Product is the only one with text labels (colour, season, usage); Fashionpedia brings real street photos plus pattern and **coverage** (modesty) attributes; PolyVore brings user-made outfits for the **compatibility** model.
- Main data risks: strong class imbalance, no colour label outside Fashion Product, ~24k duplicate images in PolyVore (split leakage), small crops in Fashionpedia, and a large domain gap between catalogue shots and friperie phone photos.
- Mapping rules live in mappings/* .csv, never in code; the apply scripts stop when a source value has no rule. 58 rules are flagged **REVIEW** for team decision.
- **Colour** was estimated for PolyVore and Fashionpedia with a small model trained on Fashion Product's real labels. At the confidence cut used it is 83.1% accurate on unseen Fashion Product images (it gives a colour to 49% of them). Estimated colours were kept for 52% of PolyVore items and 49% of Fashionpedia items *with usable pixels*, i.e. 52% and 32% of all rows; otherwise the colour stays empty.
- **Merged dataset**: 321,422 rows (296,921 unique pictures / crops) in one file, split **246,373 / 36,750 / 38,299** (train / val / test) with no picture shared between splits. All Phase 3 goals are done except the collection of local photos.

1. Context and goals

DressMe targets young Tunisians on a limited budget who shop mostly second-hand (friperie), often buying unlabelled items that can rarely be returned. Phases 1 (Empathize) and 2 (Ideate) are complete; the survey (57 responses) guides the data choices:

Survey finding	Consequence for the data
Black dominates 84% of wardrobes	Colour labels must separate black / navy / charcoal well; check black share per dataset
56% shop second-hand	Real friperie photos reserved for the test set
40% frustrated by fit	Keep fit / silhouette hints (Fashionpedia)
42% want “should I buy this?” advice	Need compatibility data (PolyVore outfits) and rich attributes

Phase 3 objectives

- EDA per dataset: size, class balance, missing values, colour distribution.
- Label mapping from each dataset to the unified schema.
- Merged, cleaned dataset with train / val / test splits (local photos reserved for test).
- EDA report for the Phase 3 validation (this document).

Target tech stack

React + TailwindCSS · FastAPI · MongoDB · EfficientNet (classification) · FashionCLIP (embeddings, similarity) · Gemini API (chat) · custom weighted compatibility formula.

2. Dataset inventory

All sources are used for non-commercial academic work only and are never redistributed. Sizes are what we use after cleaning (rows in data/processed/).

Source	Content	Size	Licence / terms	Access
Kaggle Fashion Product Images (Small)	catalogue shots, text labels	35,432 items (44,441 raw)	MIT (Kaggle page)	downloaded
Fashionpedia	street / runway photos, masks, attributes	160,080 worn items in 46,743 photos	annotations CC BY 4.0; images not owned by Fashionpedia (Flickr and free-licence sites)	downloaded
Maryland PolyVore (Re-PolyVore)	product shots grouped in outfits	125,910 items in 31,333 outfits	Apache-2.0 (dataset repository); images from Polyvore users	downloaded
DeepFashion2	consumer + shop photos, 13 categories, landmarks	491k images (announced)	non-commercial research only (request form)	access requested
DressCode	high-resolution garment + model pairs	53,792 pairs (announced)	non-commercial academic research, no modified redistribution (Yoox Net-a-Porter licence)	access requested
ModaNet	street photos with pixel masks (multi-item capture)	~55k images (announced)	to check on the official page	planned, not requested yet
VITON-HD	person + garment pairs (virtual try-on)	~14k pairs (announced)	to check on the official page	planned, not requested yet
Local photos (wardrobe / friperie)	our own phone photos	target 750–1,500 items + ~100 near-duplicate pairs	signed consent of every contributor (see section 12)	in progress

The collection plan named *Polyvore Outfits* (~68k outfits); we use the Maryland version (31,333 outfits), which has the same kind of user-made outfits and was available without a request.

3. Traceability: data → features → personas

Every dataset traces back to a feature, and every feature to a persona and a “How might we” question (HMW) from Phase 2, as in the Phase 3 collection plan. Features and personas without a dataset here (occasion & weather styling for Salma, the chat assistant, the style profile) use APIs, the survey or app-generated data instead.

Dataset	DressMe features it trains or tests	Personas · HMW
Fashion Product	photo capture + auto-tagging (category, colour, season, usage); trains the colour model behind colour stretch	Amira · H2; Ines · H7
Fashionpedia	multi-item photo capture (masks); pattern tagging; coverage for the modest filter of the style profile	Amira · H2; all personas
PolyVore	outfit generation, complete the look (compatibility formula)	Amira, Youssef · H1 H3
DeepFashion2 (requested)	auto-tagging robustness on consumer photos; landmarks for the fit check	Amira · H2; Rania · H5
DressCode (requested)	virtual try-on	Ines, Youssef · H7
Local photos	real test set (friperie domain) for auto-tagging and instant scan; local garments; fit-check subset; near-duplicate pairs for the duplicate warning; purchase-advisor evaluation set	Amira · H2; Nour · H4; Rania · H5; Youssef, Nour · H3 H4; Youssef, Skander · H8

4. What each dataset brings and how it is used

The features each dataset adds to the unified schema, and where the project uses it (Phase 3 pipeline and Phase 4 prototype). H&M and the shop demo are shop catalogues added in Phase 4: they are never merged into the training file. Field-by-field mapping: `reports/phase3/dataset_feature_map.md`.

Dataset	Features it brings	How the project uses it
Kaggle Fashion Product Images (Small) 60x80 px catalogue shots on white	- the only real labels for colour (baseColour → primary_colour), season and usage - category / sub_category from articleType (142 types) - pattern from keywords in productDisplayName - gender	- trains the colour model (estimate_colours.py) that fills colours for PolyVore and Fashionpedia - trains the EfficientNet classifier (category, sub_category, pattern) - FashionCLIP embeddings and the /similar search catalogue
Fashionpedia street / runway photos, several items per photo	- real-life worn items with a bounding box (cropped for the models) - pattern from the textile pattern attribute - sub_category from the nickname attribute (kaftan → traditional) - coverage (modesty, 1-5) from item length + linked sleeve / neckline parts - outfits: every photo is one outfit (outfit_id) - no colour label: colour is estimated	- trains the classifier on crops closer to real photos than catalogue shots - coverage for the modest filter of the style profile (min_coverage) - compatibility evaluation (AUC / FITB on the test photos) - street → shop retrieval check of the FashionCLIP embeddings
Maryland PolyVore (Re-PolyVore) product shots on white	31,333 user-made outfits (outfit_id + position from the file name) - category (and sub_category for some folders) from the folder name - no text, colour or pattern labels: colour is estimated, pattern predicted	- main source for outfit compatibility (outfit_split, clean test outfits for AUC / FITB) - trains the classifier (category) - FashionCLIP embeddings and the /similar search catalogue
Local photos (wardrobe / friperie) our own phone photos, one item each	- every schema field labelled by hand, including style, secondary_colour and source = wardrobe / friperie - the only real DressMe setting: friperie items and local garments (jebba, sefsari)	test set only (merge_and_split.py forces it), to measure the gap between public data and our users
H&M Personalized Fashion Recommendations shop product photos (shrunk to 320 px)	63k adult articles with category, colour, pattern (graphical_appearance_name) and usage labels - product name, description and product_code (the colours of one product); no price	shop catalogue, never training data: the H&M row of /similar ("buy something similar")
Shop demo (Zara, Bershka, Pull&Bear) shop product photos	real prices (TND), sizes and stock per colour; not mapped to the schema	frozen snapshot of shop listings (scrapping is stopped), shown by the snapshot listings connector

5. Unified label schema

Every source is converted to the same fields so the models can be trained on the merged data.

Field	Values
category	top, bottom, dress, outerwear, shoes, bag, accessory, traditional, swimwear
sub_category	58 controlled values (t-shirt, shirt, jeans, jebba, sneakers...) in sub_category_vocabulary.csv, each tied to a parent category
primary / secondary_colour	fixed 21-colour palette (colour_palette.csv)
pattern	solid, striped, checked, floral, printed
season (multi)	summer, winter, mid-season
usage (multi)	casual, formal, sport, wedding, eid, work
style (multi)	classic, streetwear, modest, sporty, trendy
coverage	1 (very revealing) ... 5 (fully covered)
source	public_dataset, wardrobe, friperie

Mapping conventions

- Rules are CSV files in mappings/; the apply script fails if a source value has no rule, so nothing is silently dropped.
- Multi-value fields are joined with |.
- Unknown stays empty: we **never guess** (e.g. a product with no pattern keyword is not assumed solid).
- Ids are prefixed per dataset (fp_, fpd_, pv_); output goes to data/processed/<dataset>.csv.

The mapping scripts share two safety checks: a source value with no rule, or a rule that outputs a value outside the schema, stops the script (src/phase3/map_fashion_product.py).

→ Code excerpt: *appendix B.1*.

6. EDA — Kaggle Fashion Product Images (Small)

Metric	Value
Rows in styles.csv	44,446 (22 rows repaired: commas inside names)
Usable products (row + image)	44,441
Missing values	usage 317, season 21, baseColour 15, name 7, year 1
Image size	60×80 px, catalogue shots on white background
Labels	gender, master/sub category, 142 article types, 46 colours, season, usage, product name

What this dataset brings to DressMe

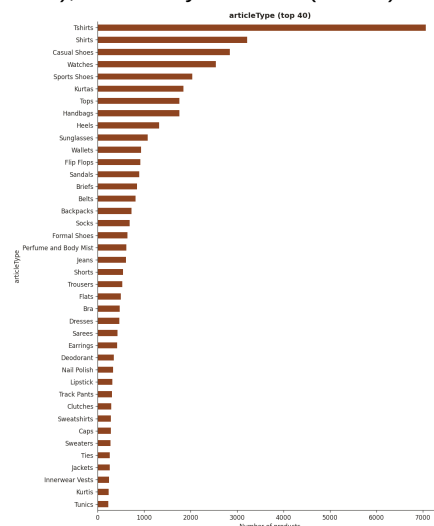
- the only real labels for colour (baseColour → primary_colour), season and usage
- category / sub_category from articleType (142 types)
- pattern from keywords in productDisplayName
- gender

How the project uses it

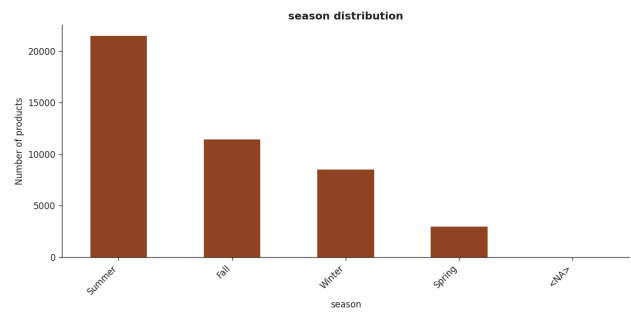
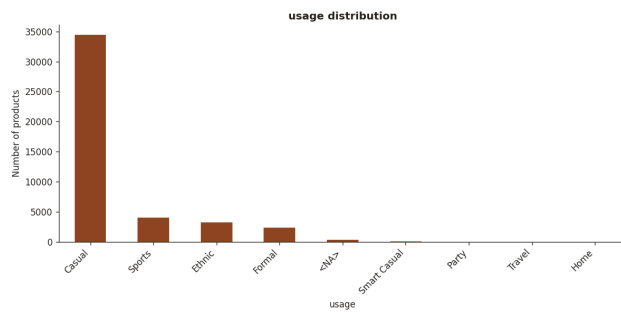
- trains the colour model (estimate_colours.py) that fills colours for PolyVore and Fashionpedia
- trains the EfficientNet classifier (category, sub_category, pattern)
- FashionCLIP embeddings and the /similar search catalogue

Class balance

- masterCategory: Apparel 48.1%, Accessories 25.4%, Footwear 20.8%, Personal Care 5.4%. **2,430 off-topic rows** (Personal Care, Home, Sporting Goods) are dropped.
- 142 article types, **72 have fewer than 50 images**: merged into coarser sub_categories during mapping. Top: Tshirts 7,069, Shirts 3,215, Casual Shoes 2,846.
- usage is dominated by Casual (77.4%); season by Summer (48.3%).

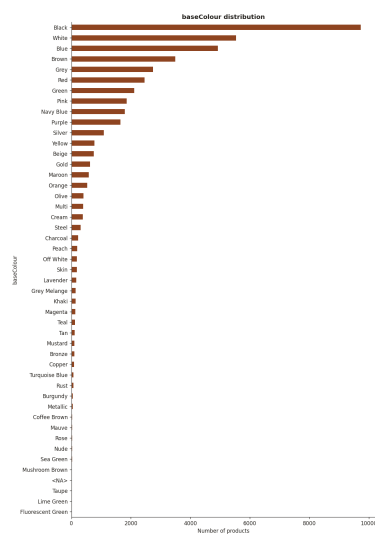


Top 40 article types



Colour distribution

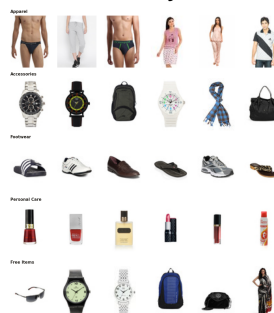
- 46 colour names, grouped into our 21-colour palette via fashion_product_colour.csv.
- Black 21.9%, White 12.5%, Blue 11.1%, Brown 7.9%, Grey 6.2% of items (section 11 discusses how this relates to the survey).



Base colour distribution

Take-aways

- Only source with text labels: the reference for colour, season and usage.
- Tiny images and studio conditions: very different from friperie phone photos, so our own photos stay in the test set.
- Pattern is read from keywords in the product name; only 21% of rows get one.



Random sample of product images

7. EDA — Fashionpedia

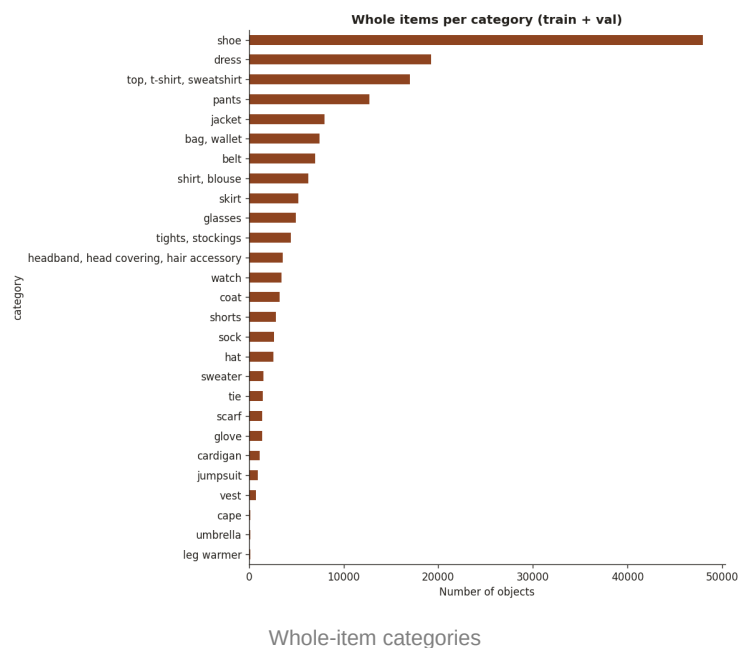
Split	Images	Objects	Whole items	Parts
train	45,623	333,401	163,060	170,341
val	1,158	8,781	4,688	4,093
test	2,044	no labels (not used)	—	—

What this dataset brings to DressMe

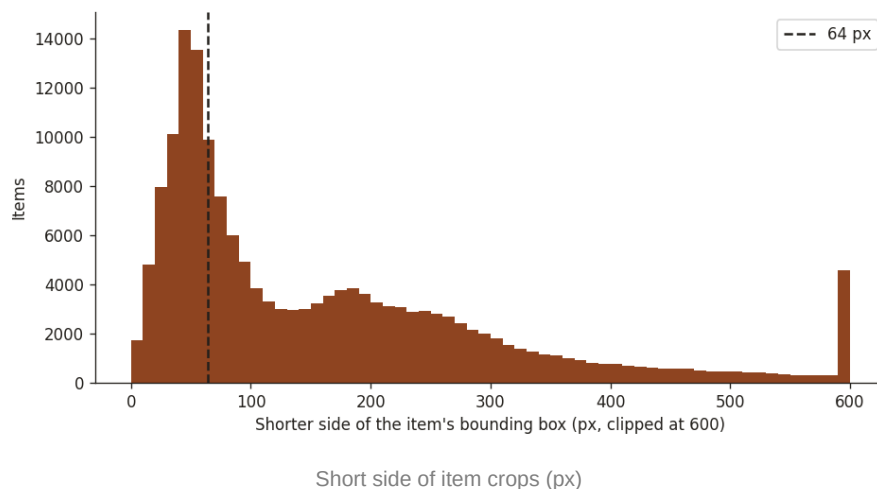
- real-life worn items with a bounding box (cropped for the models)
- pattern from the `textile` `pattern` attribute
- `sub_category` from the `nickname` attribute (kaftan → traditional)
- coverage (modesty, 1-5) from item length + linked sleeve / neckline parts
- outfits: every photo is one outfit (`outfit_id`)
- no colour label: colour is estimated

How the project uses it

- trains the classifier on crops closer to real photos than catalogue shots
- coverage for the modest filter of the style profile (`min_coverage`)
- compatibility evaluation (AUC / FITB on the test photos)
- street → shop retrieval check of the FashionCLIP embeddings
- Street and runway photos, several items per photo (median 3, max 20): each photo is one outfit. Median image 682×1024 px.
- 46 categories (27 whole items, 19 garment parts such as sleeve, neckline, pocket) and 294 attributes in 11 groups.
- **No colour label.** 53% of whole items have no attribute at all (mostly shoes and accessories); 94.1% of clothes have a textile-pattern attribute.
- Ids restart per split, so they are prefixed with the split name.

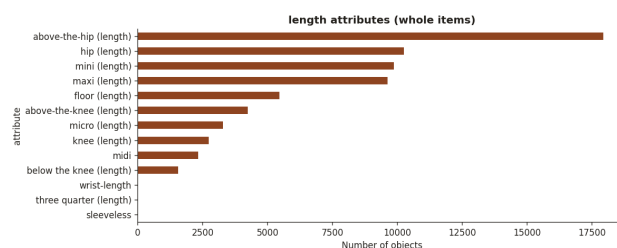
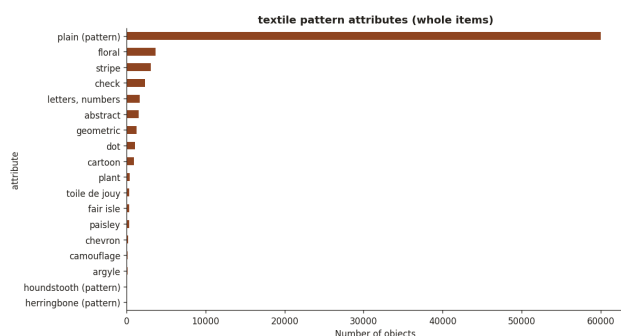


- Imbalance: shoe 28.6% (labelled one per foot), dress 11.5%, top 10.1%; cape, leg warmer, umbrella < 0.1%.
- **33.8% of item crops are shorter than 64 px** (watch 93%, sock 90%, glasses 87%): probably too small to classify.



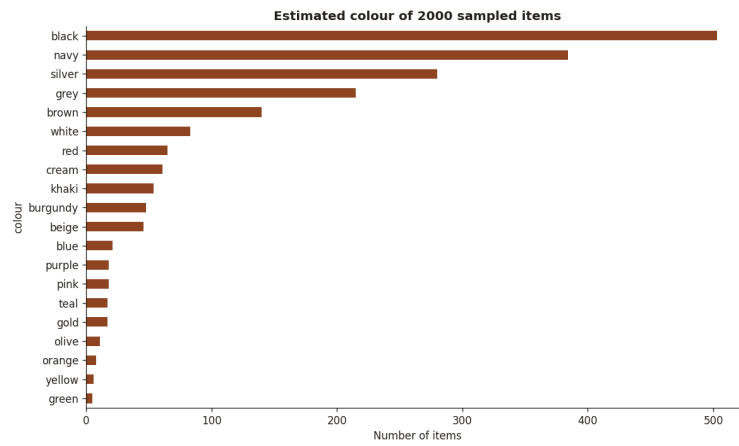
Attributes used for the schema

Fashionpedia group	Items with it	→ DressMe field
textile pattern	44.3%	pattern (plain 60k, floral 3.7k, stripe 3.1k...)
length	40.1%	coverage (with sleeve / neckline parts)
nickname	30.1%	sub_category (118 values: classic t-shirt, gown, jeans...)
silhouette	44.8%	fit / style hints (regular, tight, loose...)



Estimated colour

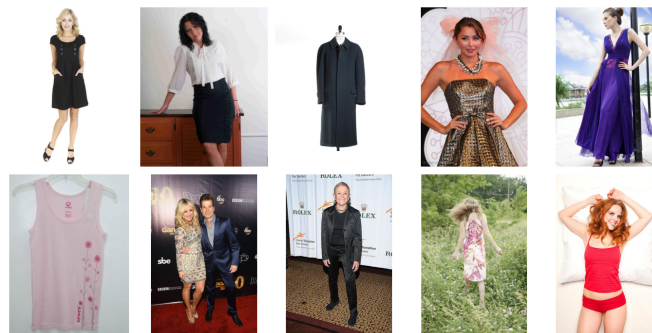
- Median pixel colour inside 2,000 random clothing masks (crop ≥ 64 px), snapped to the palette with a Lab distance.
- black 25.1%, navy 19.2%, silver 14.0%, grey 10.8%, brown 7.0%. Shadows push colours towards grey / silver, and multicolour can never be predicted this way.



Estimated colour of clothes

Colours are compared in Lab space, which is closer to human perception than RGB (src/phase3/colour_utils.py).

→ Code excerpt: appendix B.2.



Example Fashionpedia photos

8. EDA — Maryland PolyVore (Re-PolyVore)

Metric	Value
Images	126,927 in 20 category folders
Outfits	31,333 (median 4 items, max 34)
Junk files	22 (desktop.ini, a .lnk shortcut, a "- Copy.jpg")
Unlinked images	276 named .jpg (no outfit)
Exact duplicates	24,273 extra copies in 14,756 groups; 304 groups span two folders (label conflicts)
Background	white in 99% of a 500-image sample
Text / colour / split	none

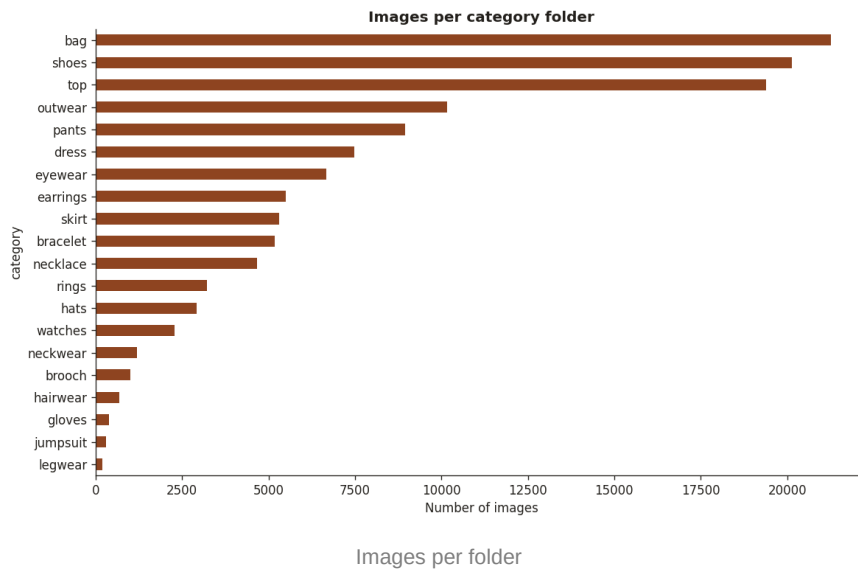
What this dataset brings to DressMe

- 31,333 user-made outfits (outfit_id + position from the file name)
- category (and sub_category for some folders) from the folder name
- no text, colour or pattern labels: colour is estimated, pattern predicted

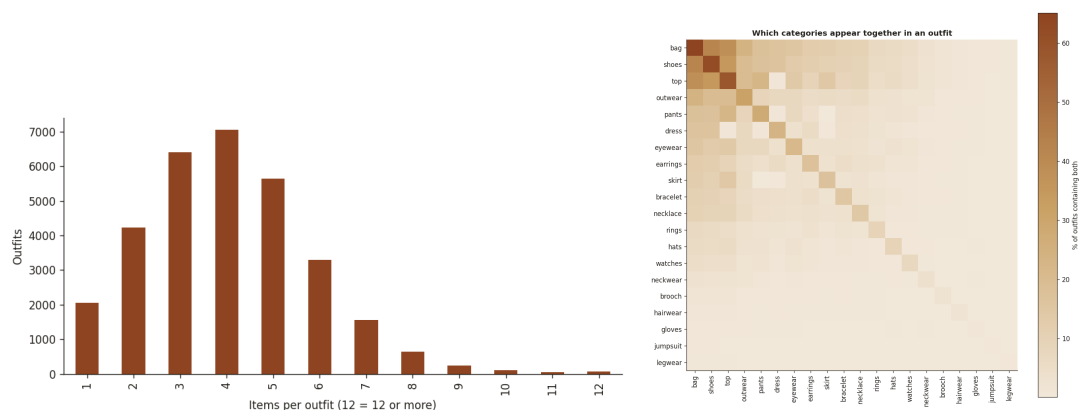
How the project uses it

- main source for outfit compatibility (outfit_split, clean test outfits for AUC / FITB)
- trains the classifier (category)

- FashionCLIP embeddings and the /similar search catalogue



- Folders are coarse: pants also holds shorts, top holds shirt-jackets, so sub_category is only filled when the folder is unambiguous (35.8% of rows).
- Accessories (jewellery, eyewear, hats...) are 26.7% of images; no traditional items.
- Outfits with clothes **and** shoes: 39.1% (12,249). 2,053 single-item outfits are useless for compatibility.



Estimated colour

- Clothes only: black 19.5%, silver 15.5%, grey 13.5%, cream 9.1%, navy 9.1%.
- Main systematic error: white / very light items land on silver. Metallic colours should probably be allowed only for shoes, bags and jewellery (team decision).

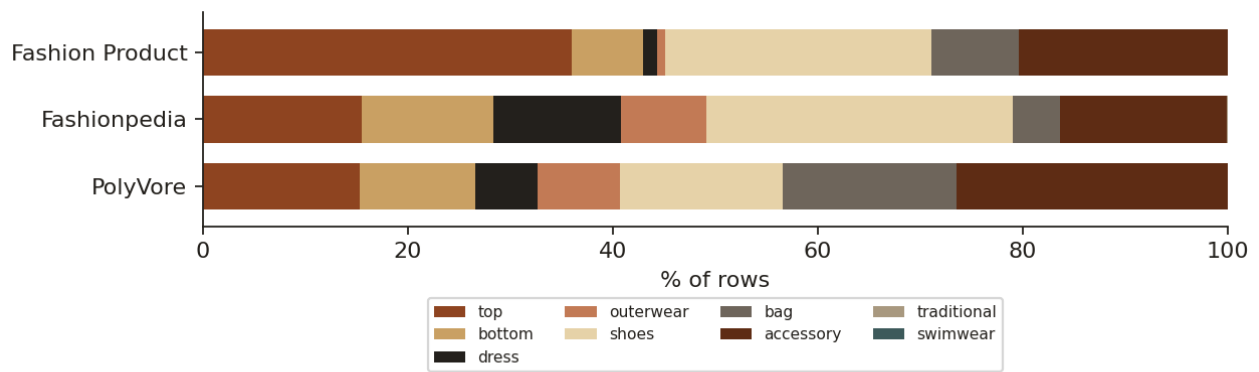


→ Code excerpt: appendix B.3.



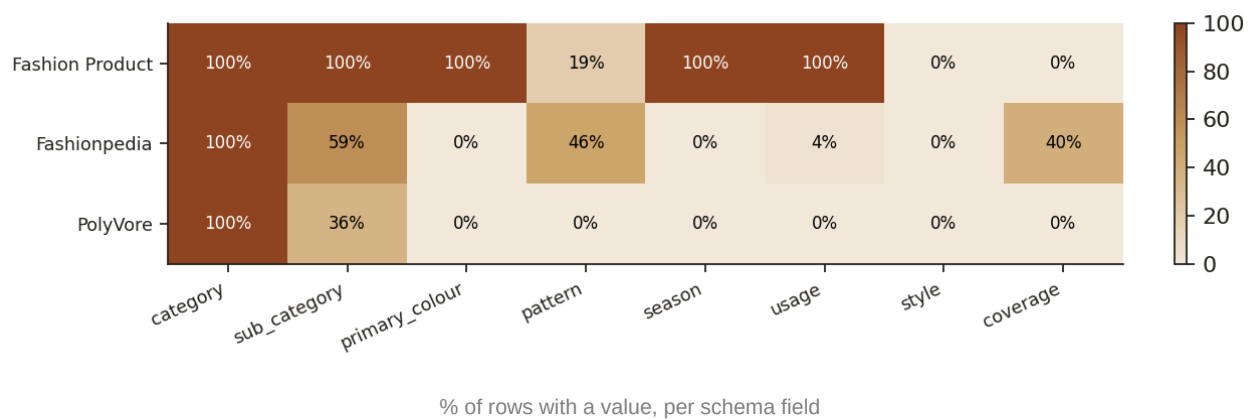
Dataset	Script	Rows out	Notes
Fashion Product	map_fashion_product.py	35,432	off-topic and Indian ethnic rows dropped; colour, season, usage from text
Fashionpedia	map_fashionpedia.py	160,080	one row per worn item, bbox to crop, outfit_id = photo
PolyVore	map_polyvore.py	125,910	folder-based labels, image_group = MD5, conflicts dropped
Total		321,422	

Category mix per source



Combined: shoes 77,237, accessory 66,532, top 56,782, bottom 37,226, bag 31,690, dress 28,164, outerwear 23,598, traditional 124, swimwear 69. Shoes and accessories dominate; dress and outerwear are the smallest clothing classes, and **traditional has no public data yet** (jebba, kaftan... must come from our own photos).

Which fields each source can fill



- **style** is empty everywhere: no public dataset labels it; it will come from the model / team annotation.
- **Colour** exists only for Fashion Product; PolyVore and Fashionpedia colours are estimated in section 10.
- **coverage** only comes from Fashionpedia (40.1% of items).

Coverage from Fashionpedia parts

Neckline and sleeve-length attributes sit on *part* objects, not on the garment. Each part is linked to the one upper garment whose box contains it; the final coverage is the **lowest** score among length, nickname, sleeve and neckline (the most revealing part decides).

→ Code excerpt: *appendix B.4*.

Coverage distribution: $5 \rightarrow 22,486 \cdot 4 \rightarrow 6,982 \cdot 3 \rightarrow 10,963 \cdot 2 \rightarrow 18,994 \cdot 1 \rightarrow 4,722$. Limitation: sleeveless items have no sleeve part, so their coverage can be overestimated when `sleeve` is not in `coverage_from`.

Pattern in Fashion Product is read from the product name, and left empty when no keyword matches:

```
def find_pattern(name, keywords):
    """Return the pattern of the first keyword (by priority) found in the product name."""
    if pd.isna(name):
        return pd.NA
    words = name.lower()
    for kw, pattern in keywords:
        if re.search(rf"\b{kw}\b", words):
            return pattern
    return pd.NA # unknown: do NOT assume solid
```

10. Colour estimation (PolyVore, Fashionpedia)

Only Fashion Product has colour labels. We first tried the EDA rule (median pixel colour snapped to the nearest palette colour) and measured it against Fashion Product's real labels: it was right only **25–36%** of the time. Black fabric photographs as dark blue-grey and became *navy*, white in soft shadow became *cream* or *silver*, denim became *grey*. So the colour is **learned** instead:

- Features: a 6×6×6 colour histogram in Lab space plus a few percentiles of the item's pixels. Product shots: white background removed (near-white regions touching the border). Fashionpedia: pixels inside the item's segmentation mask only.
- Model: gradient boosting trained on Fashion Product images of the **train** split, evaluated on its val + test images (never seen during training).
- **Never guess**: the colour stays empty when the model's confidence is below the cut, or when a Fashionpedia item is too small (crop < 64 px) or has no polygon mask. `colour_confidence` is kept so a stricter cut can be used later, and `colour_source` = label / estimated.
- Gold and silver are not allowed for clothes (a team decision): the model's next best colour is used instead.

Accuracy on unseen Fashion Product images

All predictions: 64.5% on 7100 images.

Minimum confidence	Colour filled	Accuracy
0.0	100%	64.5%
0.4	87%	69.5%
0.5	75%	74.1%
0.6	62%	79.1%
0.7 ← used	49%	83.1%

Accuracy per true colour, above the confidence cut:

Colour	Images	Accuracy	Colour	Images	Accuracy
black	946	88%	silver	105	76%
blue	422	82%	orange	50	78%
white	369	79%	gold	44	75%
brown	268	88%	olive	32	84%
red	253	85%	burgundy	26	65%
green	224	88%	beige	23	35%
grey	180	64%	cream	19	53%
pink	157	85%	teal	9	56%
purple	149	96%	multicolour	4	0%
navy	116	68%	khaki	2	100%
yellow	107	96%			

Fashion Product photos are tiny (60×80) and often show a model, so these numbers are a rough guide. PolyVore (product shots on white) should behave similarly; Fashionpedia (street photos, shadows) is probably less accurate: see the check grids.

Accuracy on hand-labelled PolyVore and Fashionpedia items

Random val / test items (crops ≥ 64 px), labelled by hand without seeing the model's answer; “unsure” labels are left out. 95% confidence interval in brackets.

Dataset	Min. confidence	Colour kept	Accuracy (95% CI)
Fashionpedia	0.0	100% (195)	49.7% (43–57)
Fashionpedia	0.5	78% (153)	56.9% (49–64)
Fashionpedia	0.6	64% (124)	61.3% (52–69)
Fashionpedia	0.7	44% (85)	65.9% (55–75)
PolyVore	0.0	100% (200)	63.0% (56–69)
PolyVore	0.5	80% (159)	71.7% (64–78)
PolyVore	0.6	66% (133)	74.4% (66–81)
PolyVore	0.7	52% (103)	76.7% (68–84)

Most frequent mistakes (true → predicted, all confidences): black → navy: 20; black → brown: 19; navy → blue: 8; beige → brown: 6; white → silver: 4. Black read as navy or brown is the main error, which matters because black dominates our users' wardrobes.

Result

Dataset	All rows	Items with usable pixels	Colour kept	% of usable items	% of all rows
PolyVore	125,910	125,910	65,795	52.3%	52.3%
Fashionpedia	160,080	104,301	51,516	49.4%	32.2%

Usable pixels: every PolyVore picture; for Fashionpedia only items with a polygon mask and a crop of at least 64 px (the others get no estimate at all).





Fashionpedia: estimated colour of item crops (confidence)

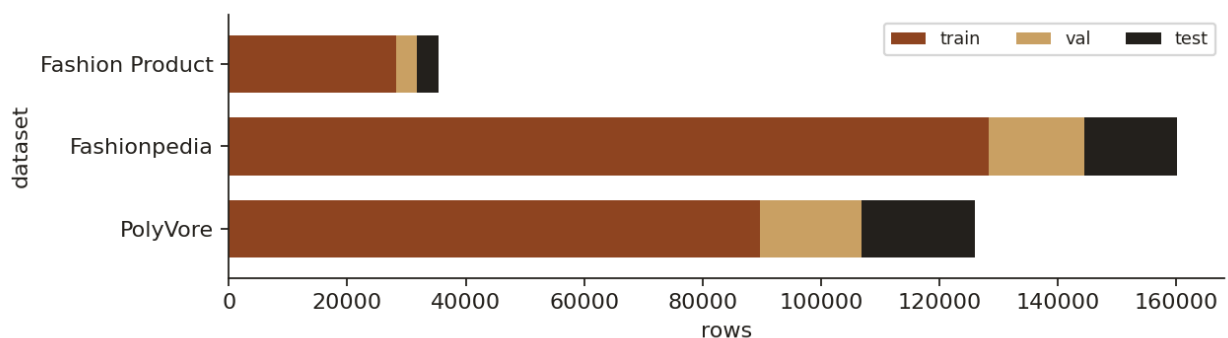
The features, the metal rule and the confidence cut are in `src/phase3/estimate_colours.py`.

→ Code excerpt: [appendix B.5](#).

11. Merged dataset and train / val / test splits

`src/phase3/merge_and_split.py` joins the three processed files and the colour estimates into `data/processed/dressme.csv`: **321,422 rows**, of which 24,501 are extra copies of the same picture (`duplicate = True`, to drop for classification).

Dataset	train	val	test	total
Fashion Product	28,331	3,477	3,624	35,432
Fashionpedia	128,322	16,109	15,649	160,080
PolyVore	89,720	17,164	19,026	125,910
Total	246,373	36,750	38,299	321,422



How the splits are made

- Fashionpedia's official test set has no labels, so our train / val / test splits for Fashionpedia are all made from its labelled train + val images.
- 80 / 10 / 10, decided by a hash of the group id: the same group always lands in the same split, even after re-running the script or adding rows.
- `split` (classification) is per picture: all copies of a picture (`image_group`) and all items of a Fashionpedia photo share one split.
- `outfit_split` (compatibility) is per outfit. In PolyVore the same product is reused in many outfits, which share other products, and so on: **61% of PolyVore rows form one chain**, so outfits and pictures cannot both be kept apart. A picture used in a test outfit goes to test, so every test outfit is made only of test pictures; `outfit_clean = False` marks outfits that share a picture with another split.

- Local photos (source = wardrobe / friperie) always go to test. The script stops if a picture ends up in two splits, or a test outfit contains a non-test picture.

→ Code excerpt: *appendix B.6*.

Class balance across splits

Share of each category's unique pictures in each split: every class stays close to 80 / 10 / 10, so val and test represent all categories.

Category	unique items	train %	val %	test %
shoes	73,472	79.5	10.3	10.2
accessory	59,604	78.3	10.9	10.8
top	52,489	79.0	10.5	10.5
bottom	34,052	78.7	10.7	10.7
bag	27,941	77.9	11.3	10.8
dress	27,440	80.0	10.0	10.0
outerwear	21,730	78.9	10.6	10.5
traditional	124	85.5	8.9	5.6
swimwear	69	78.3	14.5	7.2

Outfits for the compatibility model

Dataset	outfit_split	clean outfits	sharing a picture
Fashionpedia	test	4,575	0
Fashionpedia	train	37,495	0
Fashionpedia	val	4,681	0
PolyVore	test	1,064	2,013
PolyVore	train	16,674	8,345
PolyVore	val	1,090	2,128

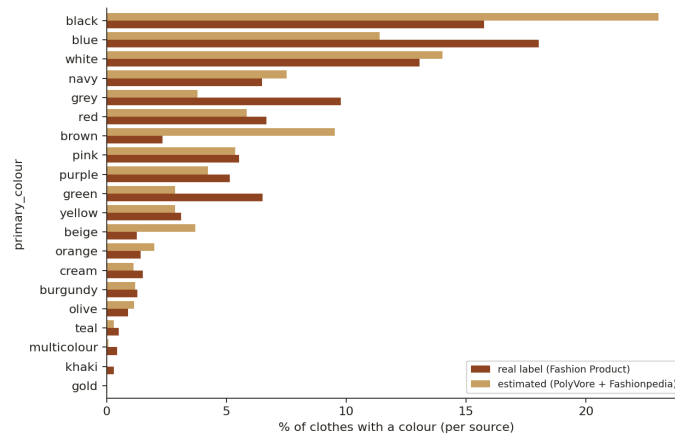
Fashionpedia outfits (one photo each) are all clean. For PolyVore, a strict evaluation should use only the clean val / test outfits; the others are fine for a “non-disjoint” evaluation, as in the original Polyvore benchmark.

Missing values in the final file



- primary_colour is filled for 47% of unique pictures / crops and 48% of all 321,422 rows (real label or estimate); the rest stays empty on purpose.
- style is still empty everywhere, and season / usage come almost only from Fashion Product: these fields will need model predictions or team annotation.

Colour distribution



Clothes only: real labels vs estimated colours

Black is 15.7% of labelled clothes and 23.0% of clothes with an estimated colour. Our survey found that black dominates 84% of wardrobes. That is a share of wardrobes, not of items, so the numbers cannot be compared directly: black is well represented in every source, but the public data are probably more colourful than our users' wardrobes: the local test photos will show how much this matters.

12. Local data collection (in progress)

Our own photos are the only data from the real DressMe setting: phone photos of second-hand and wardrobe items in Tunisia. They are used **only for the test set** (source = wardrobe / friperie; the split script enforces it) and are never committed or shared.

Collection stream	Target	Protocol
Team and volunteer wardrobes	500–1,000 items	The six team members and survey volunteers photograph their clothes in real conditions (phone camera, home lighting, hanger / bed / floor).
Friperie sessions	150–300 items	Photos at the rack with the shop owner's permission, reproducing Nour's 30-second scenario.
Local garments	50–100 items	Traditional and occasion pieces (jebba, kaftan, sefsari...) from family wardrobes.
Fit-check subset	50–100 items	Garment laid flat next to an A4 sheet for scale, plus tape measurements (chest, waist, length, inseam).
Near-duplicate pairs	~100 pairs	Similar items (same type and colour) labelled duplicate / not duplicate.

Consent and ethics

- Signed consent form for every contributor; photos can be withdrawn at any time.
- Faces blurred or cropped; no personal data beyond an anonymous contributor ID.
- No scraping of Instagram or Facebook sellers without explicit permission.
- Photos kept in the team's private storage and used only for this academic project.

Still to fix	Plan
Labelling (who labels, double-check rule)	[to fill]
Planned dates (collection plan: week 2 of the Phase 3 timeline)	[to fill]

13. Team decisions (former REVIEW rows)

Topic	Current rule	Alternative
Indian ethnic wear (Fashion Product, Fashionpedia)	decided: removed (kurtas, kurtis, churidar, salwar, patiala, dupatta, sarees, Nehru jackets..., plus every item with usage "Ethnic")	map kurtas to tunic, Ethnic usage to formal
Swimwear (Fashion Product, Fashionpedia)	decided: new category swimwear (swimsuit, swim-shorts); swimming caps → accessory, goggles dropped	drop it
Kaftan (Fashionpedia)	decided: traditional (close to the Maghreb caftan)	dress
Herringbone (Fashionpedia)	decided: pattern left empty (a fine zigzag weave, neither checked nor solid)	checked or solid
Eyewear	decided: sunglasses + eyeglasses merged	split (Kaggle only has sunglasses)
Metallic colours	decided: not allowed for clothes (estimates only)	allowed for all items
Colour confidence cut	decided: 0.7 (83.1% accurate on Fashion Product; hand labels: 66% on Fashionpedia, 77% on PolyVore)	0.6: more colours, less accurate
PolyVore outfits sharing pictures	kept, flagged outfit_clean = False	drop them from val / test

14. Data quality risks and mitigations

Risk	Mitigation
Class imbalance (shoes, tops vs dress, outerwear; 72 rare article types)	Coarse sub_categories, class weights / resampling at training
Estimated colours are noisy	Learned model, confidence cut, colour_source / colour_confidence kept
PolyVore duplicates (~24k)	Same split per image_group; duplicate flag
Fashionpedia small crops (33.8% < 64 px)	Filter by crop size for classification; no colour estimated
Outfit leakage	outfit_split per outfit; outfit_clean for strict tests
Domain gap (catalogue vs phone photos)	Local wardrobe / friperie photos reserved for test
No traditional clothing	Collect local photos (jebba, kaftan...)
Very few swimwear items (under 100)	Collect local photos (including modest swimwear)

15. Next steps

- Collect and label local wardrobe / friperie photos (test set, traditional items).
- Phase 4: train EfficientNet on split (without duplicates), compute FashionCLIP embeddings, and fit the compatibility formula on outfit_split.
- Once FashionCLIP embeddings exist, a colour model on top of them may beat the histogram model: compare on the same Fashion Product val / test images.
- Add DeepFashion2 and DressCode when access is granted.

Appendix A — Project files

Datasets are used for non-commercial academic purposes only and are not redistributed.

Path	Content
src/phase3/eda_*.py	EDA scripts → reports/phase3/eda_*.md, reports/phase3/figures/
src/phase3/map_*.py	Mapping scripts → data/processed/*.csv
src/phase3/estimate_colours.py	Colour model → data/processed/colour_estimates.csv, reports/phase3/colour_estimation.md
src/phase3/merge_and_split.py	Merge + splits → data/processed/dressme.csv
src/phase3/colour_utils.py	Lab conversion and palette matching
src/phase3/dataset_roles.py	What each dataset brings and how it is used (section 4, the EDA reports and the notebook)
reports/phase3/dataset_feature_map.md	Field-by-field mapping of every dataset to the unified schema
mappings/*.csv	All mapping rules, palette, sub_category vocabulary
src/phase3/make_colour_labelling_sheet.py	Hand-labelling set (200 + 200 items) and labelling page → data/interim/colour_labelling/
src/phase3/evaluate_colour_labels.py	Accuracy on the hand labels → reports/phase3/colour_validation.*
src/common/plot_style.py	DressMe chart colours
src/phase3/build_phase3_report.py	Builds this PDF

Appendix B — Code excerpts

B.1

```
def check_covered(values, mapping_keys, what):
    """Stop with a clear message if a source value has no mapping rule."""
    missing = sorted(set(values.dropna()) - set(mapping_keys))
    if missing:
        sys.exit(f"ERROR: {what} values with no rule in mappings/: {missing}")

def check_allowed(values, allowed, what):
    """Stop if a mapping file uses a value that is not in the unified schema."""
    wrong = sorted(set(values) - allowed - {""})
    if wrong:
        sys.exit(f"ERROR: {what} values not in the unified schema: {wrong}")
```

B.2

```
def rgb_to_lab(rgb):
    """Convert sRGB colours (N x 3, 0-255) to CIE Lab."""
    c = np.asarray(rgb, dtype=float) / 255.0
    c = np.where(c > 0.04045, ((c + 0.055) / 1.055) ** 2.4, c / 12.92) # linear RGB
    m = np.array([[0.4124, 0.3576, 0.1805],
                  [0.2126, 0.7152, 0.0722],
                  [0.0193, 0.1192, 0.9505]])
    xyz = c @ m.T / np.array([0.95047, 1.0, 1.08883]) # normalise to D65 white
    f = np.where(xyz > 0.008856, np.cbrt(xyz), 7.787 * xyz + 16 / 116)
    return np.stack([116 * f[:, 1] - 16, 500 * (f[:, 0] - f[:, 1]),
                    200 * (f[:, 1] - f[:, 2])], axis=1)

class Palette:
    def nearest(self, rgb):
        """Name of the palette colour closest to one RGB colour."""
        dist = np.linalg.norm(self.lab - rgb_to_lab([rgb]), axis=1)
        return self.names[dist.argmin()]
```

B.3

```
# label conflicts: same picture, different unified categories -> drop all copies.
labels = df.groupby("md5")[["category", "sub_category"]].nunique()
conflict = labels[(labels["category"] > 1) | (labels["sub_category"] > 1)].index
df = df[~df["md5"].isin(conflict)]

out = pd.DataFrame({
    "id": "pv_" + df["folder"] + "_" + df["file"].str.removesuffix(".jpg"),
    "category": df["category"],
    ...
    "image_group": df["md5"], # same value = same picture: keep in one split
})
```

B.4

```
def link_parts(anns, part_id):
    """Link each part (sleeve or neckline) to the one garment that contains it."""
    parts = anns[anns["category_id"] == part_id]
    garments = anns[anns["category_id"].isin(UPPER_IDS)]
    pairs = parts.merge(garments, on="image_id", suffixes=("", "_g"))

    # intersection of the two boxes, as a share of the part's box area
    x1 = np.maximum(pairs["bbox_x"], pairs["bbox_x_g"])
    y1 = np.maximum(pairs["bbox_y"], pairs["bbox_y_g"])
    x2 = np.minimum(pairs["bbox_x"] + pairs["bbox_w"], pairs["bbox_x_g"] + pairs["bbox_w_g"])
    y2 = np.minimum(pairs["bbox_y"] + pairs["bbox_h"], pairs["bbox_y_g"] + pairs["bbox_h_g"])
    inter = (x2 - x1).clip(lower=0) * (y2 - y1).clip(lower=0)
    pairs["inside"] = inter / (pairs["bbox_w"] * pairs["bbox_h"]).clip(lower=1)

    pairs = pairs[pairs["inside"] >= MIN_INSIDE]
    n_owners = pairs.groupby("ann_id")["ann_id_g"].transform("size")
    return pairs[n_owners == 1] # keep only unambiguous owners
```

B.5

```
def features(rgb_pixels):
    """Fixed-length colour description of a set of pixels (N x 3, RGB)."""
    lab = rgb_to_lab(rgb_pixels)
    hist, _ = np.histogramdd(lab, bins=(L_EDGES, AB_EDGES, AB_EDGES))
    hist = hist.ravel() / len(lab)
    pct = np.percentile(lab, [10, 25, 50, 75, 90], axis=0).ravel()
    return np.concatenate([hist, pct, lab.mean(axis=0), lab.std(axis=0)])

def predict(clf, X, categories):
    """Best colour per row; metallic colours are skipped for clothes."""
    proba = clf.predict_proba(X)
    classes = np.array(clf.classes_)
    no_metal = np.isin(np.asarray(categories), list(METAL_FREE))
    proba[np.ix_(no_metal, np.isin(classes, list(METALS)))] = 0
    proba = proba / proba.sum(axis=1, keepdims=True)
    best = proba.argmax(axis=1)
    return classes[best], proba[np.arange(len(best)), best]

# the colour stays empty when the model is not sure enough
df["primary_colour"] = np.where(conf >= MIN_CONFIDENCE, colour, "")
```

B.6

```
def bucket(key):
    """Deterministic split for a group id: same key -> same split, every run."""
    n = int(hashlib.md5(f"{SEED}:{key}".encode()).hexdigest()[:8], 16) % 100
    if n < SHARES["train"]:
        return "train"
    if n < SHARES["train"] + SHARES["val"]:
        return "val"
    return "test"

# pictures: strictest split among their outfits (test > val > train), else own hash
in_outfits = (df[has_outfit].assign(r=df["outfit_split"].map(rank))
               .groupby("image_group")["r"].max().map(name))
group_split = dict(zip(groups, groups.map(bucket)))
group_split.update(in_outfits.to_dict())
df["split"] = df["image_group"].map(group_split)
```